

CSE-346, Numerical Analysis,
Fall, 2026
SMCS, IBA Karachi

The faculty members are requested to fill in all the information in the specified sections of this document.

Program	BS(CS)
Department Offering	Department of Mathematics
Faculty	Dr. M. Sami Siddiqui
Email	ssiddiqui@iba.edu.pk
Counselling hours	04:55 to 05:25 Fri & Sat

Class Details	
Session Days and Timings	Fri & Sat 05:30 PM to 06:45PM
Class Location	MTC-18
Credit Hours	3
Class UMS Number	102005

Course Description
This course provides a rigorous yet computationally grounded introduction to numerical analysis for computer science students. It develops the classical theoretical core of the subject; floating-point error analysis, root-finding for nonlinear equations (bisection, fixed-point iteration, Newton-type methods, and polynomial root-finding), polynomial and spline interpolation, numerical differentiation and integration, and direct and iterative methods in numerical linear algebra (including eigenvalue approximation); for the numerical solution of initial-value problems in ordinary differential equations, alongside a mandatory hands-on Python programming component. Students implement every major algorithm from first principles and benchmark their code against MathLab/NumPy/SciPy library routines. Optional extension sessions extend this core to nonlinear systems, the QR algorithm and singular value decomposition, and gradient-based optimization, connecting classical techniques to modern computational and machine-learning applications such as PCA and automatic differentiation. By the end of the course, students will be able to select, implement, and critically evaluate numerical methods for real scientific and engineering computing problems.

Course Learning Outcomes (CLOs)
By the end of this course, students will be able to: 1. Analyze sources of rounding and truncation error and the convergence behavior of numerical algorithms, and apply bracketing (bisection), fixed-point, and Newton-type methods to solve nonlinear equations in one variable, including cases of multiple roots and polynomial root-finding.

2. Apply and implement direct methods (Gaussian elimination, LU factorization with pivoting) and classical iterative methods (Jacobi, Gauss–Seidel, SOR, conjugate gradient) to solve linear systems, and apply the power method and Householder's method to approximate eigenvalues.
3. Construct and evaluate polynomial and piecewise-polynomial (spline) interpolants, and derive and apply numerical differentiation and integration schemes (Richardson extrapolation, Newton–Cotes rules, Romberg integration, Gaussian quadrature), assessing their accuracy.
4. Derive, implement, and analyze the stability and convergence of numerical methods (Euler's method, Taylor-series methods, Runge–Kutta methods, and multistep methods) for initial-value problems in ordinary differential equations.
5. Implement the numerical methods above in MatLab or Python (NumPy/SciPy), validate them against analytical solutions or library routines, and apply eigenvalue-based (QR algorithm, SVD) and optimization-based techniques to problems in computational science and machine learning.

Teaching and Learning Methodology

The course combines twice-weekly lecture sessions with a mandatory hands-on practice in class. Lecture sessions develop the mathematical derivation, error analysis, and convergence theory behind each method; companion coding sessions require students to implement the same methods from first principles in Python (NumPy, SciPy, and Matplotlib, in Jupyter notebooks) before comparing their implementation against library routines. Instruction blends traditional derivation, live coding demonstrations, in-class problem solving, and guided programming assignments.

Teaching Schedule

Session #	Readings	Mapping to Course Learning Outcomes	Session Topic
	Reference of the Reading material	Faculty must specify the course learning outcome (CLO) students must expect to achieve from the respective topic	Mention the Topic
1	Burden & Faires, §1.1–1.2	CLO1	Review of calculus
2	Burden & Faires, §1.3	CLO1, CLO5	Error Analysis & Computers
3	Burden & Faires, §2.1	CLO1	Root-finding: the bisection method
4	Burden & Faires, §2.2	CLO1	Fixed-point iteration
5	Burden & Faires, §2.1–2.2	CLO1, CLO5	Convergence analysis of bisection and fixed-point and their implementations

6	Burden & Faires, §2.3	CLO1	• Newton's method, the secant method,
7	Burden & Faires, §2.4–2.5	CLO1, CLO5	• Error analysis for iterative methods; Lab: implement bisection & Newton in Python
8	Burden & Faires, §2.5	CLO1	• Multiplicity of roots and modified Newton's Method
9	Burden & Faires, §2.6	CLO1, CLO5	• Zeros of polynomials and Muller's Method
10	Burden & Faires, §3.1–3.2	CLO3	• Lagrange interpolation and Neville's method
11	Burden & Faires, §3.3	CLO3	• Forward and divided differences
12	Burden & Faires, §3.4–3.5	CLO3	• Hermite interpolation and divided difference and Cubic Spline
13	Burden & Faires, §3.5	CLO3	• Intro to splines and cubic spline
14	Burden & Faires, §3.5	CLO3, CLO5	• Cubic spline interpolation and implementation in Python
Mid Term			
15	Burden & Faires, §4.1–4.2	CLO3	• Numerical differentiation; Richardson extrapolation
16	Burden & Faires, §4.3	CLO3	• Numerical integration: Trapezoidal and Simpson's rules
17	Burden & Faires, §4.4–4.5	CLO3	• Composite rules and Romberg integration

18	Burden & Faires, §4.6–4.7	CLO3, CLO5	Gaussian quadrature; Lab: quadrature routines in Python
19	Burden & Faires, §5.1–5.2	CLO4	Theory of initial-value problems; Euler's method
20	Burden & Faires, §5.3–5.4	CLO4, CLO5	Higher-order Taylor methods and Runge–Kutta methods; Lab: implement RK4 and compare with <code>scipy.integrate</code>
21	Burden & Faires, §5.6–5.7	CLO4	Multistep methods and variable step-size methods
22	Burden & Faires, §5.10–5.11	CLO4	Stability; stiff differential equations
23	Burden & Faires, Ch. 6	CLO2	Direct methods for linear systems: Gaussian elimination and LU factorization
24	Burden & Faires, Ch. 6	CLO2, CLO5	Pivoting strategies and special matrix types (symmetric, positive-definite, band); Lab: implement LU factorization & condition-number estimation (Programming Assignment 4 due)
25	Burden & Faires, §7.1–7.2	CLO2	Vector/matrix norms and conditioning, eigenvalues
26	Burden & Faires, §7.3	CLO2	the Jacobi and Gauss–Seidel iterative methods
27	Burden & Faires, §7.4	CLO2	Relaxation (SOR) techniques and the conjugate gradient method
28	Burden & Faires, Ch. 9	CLO2	Approximating eigenvalues: the power method and Householder's method
optional	Burden & Faires, §9.5–9.6	CLO2, CLO5	The QR algorithm and singular value decomposition;

optional	Burden & Faires, §9.5-9.6	CLO2, CLO5	• Lab: SVD applications — image compression and PCA, with a note on randomized SVD for large-scale/ML settings
optional	Burden & Faires, Ch. 10	CLO1, CLO2	• Nonlinear systems: fixed points for functions of several variables; Newton's method for systems
optional	Burden & Faires, Ch. 10	CLO2, CLO5	• Steepest-descent optimization; Lab: gradient descent from scratch vs. automatic differentiation (autograd/JAX)
Final			

Textbook and Pre-Course Reading Material

Required Text/Reading:

Richard L. Burden, J. Douglas Faires & Annette M. Burden, Numerical Analysis, 9/10th Edition, Cengage, 2016.

Recommended Books:

Timothy Sauer, Numerical Analysis, 3rd ed., Pearson, 2018.

Michael T. Heath, Scientific Computing: An Introductory Survey, 2nd ed., SIAM, 2018.

Lloyd N. Trefethen & David Bau III, Numerical Linear Algebra, SIAM, 1997.

William H. Press et al., Numerical Recipes: The Art of Scientific Computing, 3rd ed., Cambridge University Press, 2007.

Useful Websites/materials (if applicable):

NumPy / SciPy documentation — numpy.org, scipy.org

MIT OpenCourseWare 18.330 / 18.335 — Introduction to Numerical Analysis / Numerical Methods

Stanford CME108 / MATH114 — Introduction to Scientific Computing

Prerequisite Skills and Knowledge to take this Course

Calculus I & II, Linear Algebra, and an introductory programming course. Students should be comfortable writing basic MatLab / Python scripts (loops, functions, arrays/NumPy).

Assessment	Weight
Programming Assignments	10
Quizzes	15
Project & Report	05
Presentation & Viva	05
Midterm Examination	25
Final Examination	40
Total	100
Make-up Exam	

Make-up midterm examinations are granted only for documented medical emergencies or university-approved absences, and must be requested in writing within 3 working days of the missed exam together with supporting documentation. Approved make-ups may be conducted in an alternative format at the instructor's discretion. No make-up quizzes are given; the lowest quiz score is dropped instead. Make-up final examinations follow IBA's official examination policy.

Technology & Innovation

Python 3 (NumPy, SciPy, Matplotlib) in Jupyter notebooks is the primary computing environment for all labs and assignments. Students use Git/GitHub for version control and assignment submission. The optimization unit(optional) introduces automatic differentiation via a modern autodiff library (e.g., PyTorch autograd or JAX), connecting classical gradient-based numerical optimization to how it is implemented in current machine-learning frameworks. Students regularly benchmark their own from-scratch implementations against NumPy/SciPy's built-in routines to build intuition for when and why library code outperforms naive implementations.

Experiential Learning Exposure(s)

Faculty should state the hands-on learning opportunities included in the course (e.g., projects, simulations, fieldwork, industry engagement, etc).

Social Contribution / Impact

This course contributes to **SDG 4** (Quality Education) by building rigorous, transferable computational and quantitative-reasoning skills, and to **SDG 9** (Industry, Innovation and Infrastructure) by equipping students with the numerical and scientific-computing foundations that underpin engineering simulation, data science, and machine-learning infrastructure used across industry and research.

Use of Artificial Intelligence (AI)

Faculty should specify the if usage of Generative AI is permitted for the course and/or individual assessments. Where the use of AI is permitted, faculty should clearly Generative AI tools (e.g., ChatGPT, Claude, GitHub Copilot) may be used to help understand concepts, debug code, or explore alternative approaches while completing Python assignments and the computational project, but may not be used to generate a complete assignment solution or project report submitted as the student's own work. Any use of AI tools must be disclosed in a short note at the end of the submission, stating which tool was used and for what purpose. Students remain fully responsible for the correctness, originality, and academic integrity of all submitted work, and may be asked to explain or reproduce any part of their code or solution during grading or in person. acceptable uses and any applicable disclosure or acknowledgment requirements. Faculty should also communicate that students remain responsible for the accuracy, quality, and integrity of all submitted work.

Academic Conduct

IBA policy

Attendance Policy

IBA policy

Plagiarism Policy

IBA policy

Withdrawal Policy

IBA policy